

2026-05-08-p1-governance-propagation

Phase 1 — Governance Propagation — Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Propagate anti-bluff anchor (Article XI §11.9) + CONST-042/043 to all 60+ submodules. Owned-by-us repos get full cascade to CONSTITUTION.md/CLAUDE.md/AGENTS.md. Third-party repos get .helix-governance marker.

Architecture: Inventory all submodules → extract canonical anchor text → apply to owned-by-us submodules → apply markers to third-party → update verification script → push all deepest-first → verify.

Tech Stack: bash, git, sed

Spec: docs/superpowers/specs/2026-05-08-helixcode-zero-bluff-completion-design.md

File Structure Map

docs/improvements/submodule_governance_inventory.txt	– create
docs/improvements/submodule_governance_state.txt	– create
docs/improvements/submodule_owned.txt	– create
docs/improvements/submodule_third_party.txt	– create
docs/improvements/anti_bluff_anchor.txt	– create
docs/improvements/anti_bluff_anchor.sha256	– create
<owned-submodule>/CONSTITUTION.md	– modify or create
<owned-submodule>/CLAUDE.md	– modify or create
<owned-submodule>/AGENTS.md	– modify or create
<third-party-submodule>/.helix-governance	– create (~40)
scripts/verify-governance-cascade.sh	– modify

Task P1-01: Inventory all 60+ submodules

Files: - Create: docs/improvements/submodule_governance_inventory.txt - Create: docs/improvements/submodule_governance_state.txt - Create: docs/improvements/submodule_owned.txt - Create: docs/improvements/submodule_third_party.txt



Step 1: Generate full submodule list

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git submodule status | awk '{print $2}' | sort > docs/
    improvements/submodule_governance_inventory.txt
count=$(wc -l < docs/improvements/
    submodule_governance_inventory.txt)
echo "Total: $count submodules" >> docs/improvements/
    submodule_governance_inventory.txt
```



Step 2: Classify owned-by-us vs third-party by git URL

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
while read sm; do
    [ -z "$sm" ] && continue
    url=$(git config -f .gitmodules --get submodule.$sm.url 2>/dev/
        null)
    echo "$sm | $url"
done < <(head -n -1 docs/improvements/
    submodule_governance_inventory.txt) > /tmp/sm_urls.txt

grep -E "HelixDevelopment|vasic-digital" /tmp/sm_urls.txt | sort
    > docs/improvements/submodule_owned.txt
grep -v -E "HelixDevelopment|vasic-digital" /tmp/sm_urls.txt |
    sort > docs/improvements/submodule_third_party.txt
echo "Owned: $(wc -l < docs/improvements/submodule_owned.txt)"
    >> docs/improvements/submodule_governance_inventory.txt
echo "Third-party: $(wc -l < docs/improvements/
    submodule_third_party.txt)" >> docs/improvements/
    submodule_governance_inventory.txt
```



Step 3: Check which governance files exist in each owned-by-us submodule

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
while read line; do
    sm=$(echo "$line" | awk -F' | ' '{print $1}')
    [ -z "$sm" ] && continue
    base="$PWD/$sm"
    echo "=== $sm ==="
    for f in CONSTITUTION.md CLAUDE.md AGENTS.md; do
        [ -f "$base/$f" ] && echo "    EXISTS: $f" || echo "    MISSING:
            $f"
    done
done < docs/improvements/submodule_owned.txt > docs/improvements/
    submodule_governance_state.txt
```



Step 4: Commit inventory

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add docs/improvements/submodule_governance_*
git commit -m "chore(P1-T01): inventory all 60+ submodule
governance states"
```

Phase: 1 Task: P1-T01

Evidence: docs/improvements/submodule_governance_state.txt"

Task P1-02: Extract canonical anti-bluff anchor text

Files: - Create: docs/improvements/anti_bluff_anchor.txt - Create: docs/improvements/anti_bluff_anchor.sha256



Step 1: Extract the anti-bluff §11.9 section from root CONSTITUTION.md

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
cat > docs/improvements/anti_bluff_anchor.txt << 'ANCHOREOF'
Article XI §11.9 – Anti-Bluff Forensic Anchor

> Verbatim user mandate: "We had been in position that all tests
do execute
> with success and all Challenges as well, but in reality the
most of the
> features does not work and can't be used! This MUST NOT be the
case and
> execution of tests and Challenges MUST guarantee the quality,
the
> completion and full usability by end users of the product!"
>
> Operative rule: The bar for shipping is not "tests pass" but
"users can
> use the feature." Every PASS in this codebase MUST carry
positive runtime
> evidence captured during execution. Metadata-only /
configuration-only /
> absence-of-error / grep-based PASS without runtime evidence are
critical
> defects regardless of how green the summary line looks. No
false-success
> results are tolerable.
```

Bluff Taxonomy

- Wrapper bluff – assertions PASS but wrapper's exit-code logic is buggy
- Contract bluff – system advertises capability but rejects it in dispatch

- Structural bluff – file exists but doesn't contain working code
- Comment bluff – comment promises behavior code doesn't have
- Skip bluff – `t.Skip()` without `SKIP-OK: #<ticket>` marker

Cascaded from root HelixCode CONSTITUTION.md.
ANCHOREOF

```
sha256sum docs/improvements/anti_bluff_anchor.txt > docs/
improvements/anti_bluff_anchor.sha256
```



Step 2: Commit

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add docs/improvements/anti_bluff_anchor.*
git commit -m
    "chore(P1-T02): extract canonical anti-bluff anchor with
    SHA-256"
```

Phase: 1 Task: P1-T02

Evidence: docs/improvements/anti_bluff_anchor.sha256"

Task P1-03: Propagate governance to owned-by-us submodules

Files: - For each owned-by-us submodule: - Modify/Create: <submodule>/CONSTITUTION.md - Modify/Create: <submodule>/CLAUDE.md - Modify/Create: <submodule>/AGENTS.md



Step 1: Create the anchor text snippets

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
BASE="$PWD"
```

```
# For CONSTITUTION.md
cat > /tmp/const_anchor.txt << 'CEOF'
```

```
## Article XI §11.9 – Anti-Bluff Forensic Anchor (Cascaded)
```

```
> Verbatim user mandate: "We had been in position that all tests
    do execute
> with success and all Challenges as well, but in reality the
    most of the
> features does not work and can't be used! This MUST NOT be the
    case and
> execution of tests and Challenges MUST guarantee the quality,
    the
> completion and full usability by end users of the product!"
>
```

```
> Operative rule: The bar for shipping is not "tests pass" but
    "users can
> use the feature." Every PASS in this codebase MUST carry
    positive runtime
> evidence captured during execution. No false-success results
    are tolerable.
```

```
### Bluff Taxonomy (cascaded from root CONSTITUTION.md)
```

```
- Wrapper bluff – assertions PASS but exit-code logic is buggy
- Contract bluff – advertises capability but rejects in dispatch
- Structural bluff – file exists but doesn't contain working code
- Comment bluff – comment promises behavior code doesn't have
- Skip bluff – t.Skip() without SKIP-OK: #<ticket> marker
```

```
CAEOF
```

```
# For CLAUDE.md / AGENTS.md
```

```
cat > /tmp/claude_agents_anchor.txt << 'CAEOF'
```

```
## Anti-Bluff and Quality Mandate
```

```
### Article XI §11.9 – Anti-Bluff Forensic Anchor
```

```
> Verbatim user mandate: "We had been in position that all tests
    do execute
> with success and all Challenges as well, but in reality the
    most of the
> features does not work and can't be used! This MUST NOT be the
    case and
> execution of tests and Challenges MUST guarantee the quality,
    the
> completion and full usability by end users of the product!"
```

```
**Operative rule:** Every PASS MUST carry positive runtime
    evidence.
```

```
No false-success results are tolerable.
```

```
**Bluff Taxonomy:** wrapper, contract, structural, comment, skip.
CAEOF
```



Step 2: Apply to all owned-by-us submodules

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
```

```
BASE="$PWD"
```

```
ANCHOR_MARKER="Article XI.*11.9"
```

```
while read line; do
```

```
    sm=$(echo "$line" | awk -F' | ' '{print $1}')
```

```

[ -z "$sm" ] && continue
sm_base="$BASE/$sm"
echo "Processing: $sm"

# CONSTITUTION.md
if [ -f "$sm_base/CONSTITUTION.md" ]; then
    if ! grep -q "$ANCHOR_MARKER" "$sm_base/CONSTITUTION.md" 2>/dev/null; then
        cat /tmp/const_anchor.txt >> "$sm_base/CONSTITUTION.md"
        echo "    Updated CONSTITUTION.md"
    else echo "    CONSTITUTION.md: already has anchor"
    fi
else
    echo "# CONSTITUTION.md — HelixCode Cascaded Governance" >
        "$sm_base/CONSTITUTION.md"
    echo "" >> "$sm_base/CONSTITUTION.md"
    cat /tmp/const_anchor.txt >> "$sm_base/CONSTITUTION.md"
    echo "    Created CONSTITUTION.md"
fi

# CLAUDE.md
if [ -f "$sm_base/CLAUDE.md" ]; then
    if ! grep -q "$ANCHOR_MARKER" "$sm_base/CLAUDE.md" 2>/dev/null; then
        cat /tmp/claude_agents_anchor.txt >> "$sm_base/CLAUDE.md"
        echo "    Updated CLAUDE.md"
    else echo "    CLAUDE.md: already has anchor"
    fi
else
    echo "# CLAUDE.md — HelixCode Cascaded Governance" >
        "$sm_base/CLAUDE.md"
    echo "" >> "$sm_base/CLAUDE.md"
    cat /tmp/claude_agents_anchor.txt >> "$sm_base/CLAUDE.md"
    echo "    Created CLAUDE.md"
fi

# AGENTS.md
if [ -f "$sm_base/AGENTS.md" ]; then
    if ! grep -q "$ANCHOR_MARKER" "$sm_base/AGENTS.md" 2>/dev/null; then
        cat /tmp/claude_agents_anchor.txt >> "$sm_base/AGENTS.md"
        echo "    Updated AGENTS.md"
    else echo "    AGENTS.md: already has anchor"
    fi
else
    echo "# AGENTS.md — HelixCode Cascaded Governance" >
        "$sm_base/AGENTS.md"
    echo "" >> "$sm_base/AGENTS.md"

```

```

    cat /tmp/claude_agents_anchor.txt >> "$sm_base/AGENTS.md"
    echo "    Created AGENTS.md"
fi
done < docs/improvements/submodule_owned.txt

```



Step 3: Verify every owned-by-us submodule now has the anchor

```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
ANCHOR_MARKER="Article XI.*11.9"
failures=0
while read line; do
    sm=$(echo "$line" | awk -F' | ' '{print $1}')
    [ -z "$sm" ] && continue
    base="$PWD/$sm"
    for f in CONSTITUTION.md CLAUDE.md AGENTS.md; do
        if [ -f "$base/$f" ] && grep -q "$ANCHOR_MARKER" "$base/$f"
            2>/dev/null; then
            echo "PASS: $sm/$f"
        else
            echo "FAIL: $sm/$f - MISSING or NO ANCHOR"
            failures=$((failures + 1))
        fi
    done
done < docs/improvements/submodule_owned.txt
echo "Failures: $failures"

```

Expected: Failures: 0



Step 4: Commit inside each owned-by-us submodule (deepest-first)

```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
while read line; do
    sm=$(echo "$line" | awk -F' | ' '{print $1}')
    [ -z "$sm" ] && continue
    sm_base="$PWD/$sm"
    if [ -d "$sm_base/.git" ] || [ -f "$sm_base/.git" ]; then
        git -C "$sm_base" add CONSTITUTION.md CLAUDE.md AGENTS.md 2>/dev/null
        if git -C "$sm_base" diff --cached --quiet 2>/dev/null; then
            echo "No changes in $sm"
        else
            git -C "$sm_base" commit -m
                "governance: cascade anti-bluff anchor Article XI 11.9
                from root HelixCode
        Phase: 1 Task: P1-T03" 2>/dev/null && echo "Committed in $sm"
            || echo "Commit failed in $sm (may be detached HEAD)"
        fi
    fi
done < docs/improvements/submodule_owned.txt

```

```
fi
fi
done < docs/improvements/submodule_owned.txt
```

Task P1-04: Create .helix-governance markers for third-party submodules

Files: - Create: <third-party-submodule>/ .helix-governance (~40 files)



Step 1: Create the marker template

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
cat > /tmp/marker.txt << 'MARKEREOF'
# Helix Governance Marker
#
# This file extends HelixCode constitutional governance to this
#   third-party
# submodule. Full governance documents:
#   https://github.com/HelixDevelopment/helix_code/blob/main/
#     CONSTITUTION.md
#   https://github.com/HelixDevelopment/helix_code/blob/main/
#     AGENTS.md
#
# Key constraints:
#   CONST-042 (No-Secret-Leak): No API keys, tokens, passwords,
#     certs, or
#   credentials may be committed. Secrets in .env files (mode
#     0600), in .gitignore.
#
#   CONST-043 (No-Force-Push): No force push, history rewrite,
#     branch deletion
#   without explicit per-operation user approval.
#
#   Article XI §11.9 (Anti-Bluff): Every test PASS MUST carry
#     positive runtime
#   evidence. No false-success results are tolerable.
#
# Bluff Taxonomy: wrapper, contract, structural, comment, skip.
#
# Generated: 2026-05-08 by HelixCode Zero-Bluff Completion
#   programme
# Do not remove — verified by verify-governance-cascade.sh
MARKEREOF
```



Step 2: Apply to all third-party submodules


```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
while read line; do
    sm=$(echo "$line" | awk -F' | ' '{print $1}')
    [ -z "$sm" ] && continue
    sm_base="$PWD/$sm"
    if [ -d "$sm_base" ]; then
        cp /tmp/marker.txt "$sm_base/.helix-governance"
        echo "Created: $sm/.helix-governance"
    else
        echo "SKIP: $sm – directory not found (submodule not
            initialized?)"
    fi
done < docs/improvements/submodule_third_party.txt

```



Step 3: Commit in third-party submodules where possible

```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
while read line; do
    sm=$(echo "$line" | awk -F' | ' '{print $1}')
    [ -z "$sm" ] && continue
    sm_base="$PWD/$sm"
    if [ -d "$sm_base/.git" ] || [ -f "$sm_base/.git" ]; then
        git -C "$sm_base" add .helix-governance 2>/dev/null
        origin_url=$(git -C "$sm_base" remote get-url origin 2>/dev/
            null || echo "")
        is_ours=$(echo "$origin_url" | grep -cE "HelixDevelopment|
            vasic-digital" || true)
        if [ "$is_ours" -gt 0 ]; then
            git -C "$sm_base" commit -m "governance: add .helix-
                governance marker (HelixCode cascade)"
        fi
    fi
done < docs/improvements/submodule_third_party.txt

```

Task P1-05: Update verify-governance-cascade.sh to cover all submodules

Files: - Modify: scripts/verify-governance-cascade.sh



Step 1: Read the current script

```

cat /run/media/milosvasic/DATA4TB/Projects/helix_code/scripts/
    verify-governance-cascade.sh

```

Note the existing structure, then replace/extend.



Step 2: Write the extended verification script

```
cat > /run/media/milosvasic/DATA4TB/Projects/helix_code/scripts/
      verify-governance-cascade.sh << 'SCRIPTEOF'
#!/bin/bash
set -euo pipefail
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
ANCHOR="Article XI.*11.9"
FAILURES=0

echo "=== Governance Cascade Verification ==="

# 1. Root governance files
for f in CONSTITUTION.md AGENTS.md; do
    if grep -q "$ANCHOR" "$ROOT/$f" 2>/dev/null; then echo "PASS:
      root/$f"
    else echo "FAIL: root/$f"; FAILURES=$((FAILURES+1)); fi
done

# 2. Owned-by-us submodules
echo ""
echo "--- Owned-by-us submodules ---"
while IFS=' |' read -r sm url; do
    [ -z "$sm" ] && continue
    for f in CONSTITUTION.md CLAUDE.md AGENTS.md; do
        if [ -f "$ROOT/$sm/$f" ] && grep -q "$ANCHOR" "$ROOT/$sm/$f"
          2>/dev/null; then
            echo "PASS: $sm/$f"
        elif [ -f "$ROOT/$sm/$f" ]; then
            echo "FAIL: $sm/$f — no anchor"; FAILURES=$((FAILURES+1))
        else
            echo "MISS: $sm/$f — file missing"; FAILURES=$((FAILURES+1))
        fi
    done
done < "$ROOT/docs/improvements/submodule_owned.txt"

# 3. Third-party submodules (require .helix-governance marker)
echo ""
echo "--- Third-party submodules ---"
while IFS=' |' read -r sm url; do
    [ -z "$sm" ] && continue
    if [ -f "$ROOT/$sm/.helix-governance" ]; then
        echo "PASS: $sm/.helix-governance"
    else
```

```

        echo "FAIL: $sm/.helix-governance – missing"; FAILURES=$((FAILURES+1))
    fi
done < "$ROOT/docs/improvements/submodule_third_party.txt"

echo ""
echo "=== Result: $FAILURES failures ==="
[ "$FAILURES" -eq 0 ] && echo "PASS" && exit 0
echo "FAIL"; exit 1
SCRIPTEOF
chmod +x /run/media/milosvasic/DATA4TB/Projects/helix_code/
scripts/verify-governance-cascade.sh

```



Step 3: Run and verify it passes

```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
bash scripts/verify-governance-cascade.sh

```

Expected: PASS with zero failures



Step 4: Commit

```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add scripts/verify-governance-cascade.sh
git commit -m "feat(P1-T05): extend verify-governance-cascade.sh
to verify all 60+ submodules

```

Phase: 1 Task: P1-T05"

Task P1-06: Push all submodule changes and update main repo pointers



Step 1: Push owned-by-us submodules to their remotes

```

cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
while IFS=' | ' read -r sm url; do
    [ -z "$sm" ] && continue
    sm_base="$PWD/$sm"
    if [ -d "$sm_base/.git" ] || [ -f "$sm_base/.git" ]; then
        echo "Pushing $sm..."
        git -C "$sm_base" push origin HEAD:main 2>&1 || \
        git -C "$sm_base" push origin HEAD:master 2>&1 || \
        echo " Note: could not push $sm (may be detached HEAD for
        specific commit)"
    fi
done < docs/improvements/submodule_owned.txt

```



Step 2: Stage submodule pointer changes in main repo

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
while IFS=' |' read -r sm url; do
    [ -z "$sm" ] && continue
    git add "$sm" 2>/dev/null || true
done < docs/improvements/submodule_owned.txt
git add docs/improvements/
```



Step 3: Commit and push main repo

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git commit -m
    "chore(P1-T06): push governance cascade, update all
    submodule pointers
```

Phase: 1 Task: P1-T06"

```
git push github main && git push gitlab main
```

Task P1-07: Final Phase 1 verification



Step 1: Run governance verification

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
bash scripts/verify-governance-cascade.sh
```

Expected: PASS



Step 2: Verify anchor SHA-256 still matches

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
sha256sum -c docs/improvements/anti_bluff_anchor.sha256
```

Expected: OK



Step 3: Commit final Phase 1 state

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add . && git commit -m "chore(P1-T07): Phase 1 complete –
    governance on all 60+ submodules
```

Phase: 1 Task: P1-T07

Evidence: verify-governance-cascade.sh PASS"

```
git push github main
```

Phase 1 Completion Checklist

☐

All owned-by-us submodules have CONSTITUTION.md/CLAUDE.md/AGENTS.md with anti-bluff anchor

☐

All third-party submodules have `.helix-governance` marker

☐

`verify-governance-cascade.sh` exits 0

☐

Anchor SHA-256 verified

☐

All changes committed and pushed

☐

Continue to Phase 2